

Imitation Learning for Optimal Thrust Control of Rocket Engines

Amir Suhail Salim

Technical University of Munich

TUM School of Computation, Information and Technology

Chair of Automatic Control Engineering (LSR)

Munich, Germany

amirsuhail.salim@tum.de

Abstract—Nonlinear model predictive control (NMPC) provides effective pressure regulation for monopropellant rocket engines but requires 10–50 ms per solve, making real-time embedded deployment infeasible. This paper applies behaviour cloning (BC), a supervised imitation learning method, to train a compact neural network (NN) policy that imitates a reactive NMPC controller. The NN maps the current chamber pressure, reference pressure, and previous valve command to a valve set-point, and is trained on 10 000 expert demonstrations collected across the operating envelope of 9–15.5 bar using zero-order hold reference trajectories. A weighted mean squared error loss prioritises active control transitions during training. The resulting 56-parameter network achieves closed-loop pressure tracking comparable to the NMPC expert, with an inference time under 1 ms — a speedup of approximately two orders of magnitude. Robustness analysis shows that the inclusion of measured chamber pressure as a network input provides implicit feedback correction, mitigating covariate shift without iterative data aggregation.

Index Terms—imitation learning, behaviour cloning, nonlinear model predictive control, neural network, rocket engine, embedded control

I. INTRODUCTION

A. Motivation

Monopropellant rocket engines are widely used in spacecraft attitude control and small satellite propulsion due to their mechanical simplicity and reliable operation. Precise regulation of chamber pressure is essential for consistent thrust output, yet the underlying dynamics are near-linear within the operating envelope, with nonlinearity arising primarily from the valve flow characteristic and pressure-dependent combustion behaviour.

nonlinear model predictive control (NMPC) provides an effective solution to this control problem. By solving a finite-horizon constrained optimisation problem at every timestep, NMPC can handle nonlinear dynamics, enforce physical constraints on the valve position and rate, and track arbitrary reference trajectories. However, this performance comes at a significant computational cost. The iterative solver invoked at every 50 ms control step requires 10–50 ms on a desktop processor — leaving no margin for embedded deployment where available compute is orders of magnitude lower. Furthermore, the nonconvex nature of the optimisation problem means the solver may fail to converge within the available time budget, making real-time guarantees difficult to provide.

imitation learning (IL) offers a practical path to bridging this gap. Rather than designing a new controller from scratch, IL learns to imitate an existing expert controller from demonstrations. The resulting policy approximates the expert’s input-output mapping while requiring only a single forward pass through a neural network at inference time — executable in under 1 ms on a standard processor.

B. Objectives

This work applies behaviour cloning (BC), the simplest form of IL, to train a neural network (NN) policy that imitates a reactive NMPC controller for a monopropellant rocket engine. The specific objectives are:

- 1) Design a data collection procedure that generates representative expert demonstrations across the full operating envelope of the engine.
- 2) Train a compact NN policy that maps the current chamber pressure, reference pressure, and previous valve command to a valve set-point.
- 3) Validate the learned policy in closed-loop simulation against the NMPC expert on unseen reference trajectories.
- 4) Analyse robustness to out-of-distribution conditions and identify the limitations of the BC approach.

II. LITERATURE REVIEW: IMITATION LEARNING STRATEGIES

A. Overview of Imitation Learning

IL is a family of machine learning methods that train a policy by learning from the behaviour of an expert, rather than by designing a reward function or solving an optimisation problem from scratch. The core idea is that an expert — a human operator, an analytical controller, or an optimisation-based solver — already knows how to behave in the environment. IL extracts this knowledge from recorded demonstrations and compresses it into a learnable policy, typically a NN.

The principal advantage of IL over pure reinforcement learning (RL) in control applications is data efficiency and stability: rather than learning from sparse scalar rewards through trial and error, the agent learns from dense, labelled input-output pairs provided directly by the expert. This makes IL particularly attractive for approximating computationally

expensive controllers such as NMPC, where the expert already guarantees constraint satisfaction and near-optimal performance, but cannot be deployed in real time.

B. Behaviour Cloning

BC is the simplest form of IL. The expert is queried across a set of states to produce labelled demonstrations, and the policy is trained by minimising a supervised loss — typically mean squared error (MSE) — between the policy output and the expert action. No interaction with the environment is required during training.

Several recent works demonstrate the effectiveness of BC for approximating NMPC across different application domains. Lee and Kang [1] apply BC to learn a NN controller for autonomous driving from NMPC demonstrations. Their analysis shows that the trained network achieves a computational speedup of approximately two orders of magnitude compared to the online NMPC solver while maintaining comparable tracking performance across standard driving scenarios. This quantifies the central motivation for using BC to approximate NMPC in real-time applications.

Kim et al. [2] extend this approach to the safety-critical task of emergency collision avoidance. They train a feedforward NN from offline NMPC data and pair it with a separate linear time-varying feedback controller to handle plant-model mismatch. Robustness to vehicle parameter changes is achieved through an input dimensionalisation step that scales the NN outputs to match the physical plant at runtime. Their results show that the BC policy closely replicates NMPC behaviour with significantly reduced inference time, demonstrating feasibility in safety-critical embedded systems.

Park et al. [3] apply BC from NMPC to optimal energy management in hybrid electric vehicles. Using 7,500 expert demonstrations, they train a three-layer NN to approximate the NMPC power distribution policy. Uniquely, they validate the trained controller on real embedded hardware via process-in-the-loop testing on an ARM Cortex-A9 processor, confirming practical deployability. Their results show that the NN controller matches NMPC fuel efficiency while satisfying the real-time compute budget.

C. Advanced Imitation Learning Variants

Beyond standard BC, more sophisticated IL methods have been proposed that address the fundamental limitation of BC — namely that the trained policy is not guaranteed to perform well when it encounters states outside the training distribution. This phenomenon, known as covariate shift, arises because the training distribution is determined by the expert, whereas at deployment the policy visits states determined by its own behaviour; small errors can compound over time, driving the system into regions where the policy has never been trained.

Cao and Gopaluni [4] propose an optimize-and-train framework that integrates data generation and NN training into a single large-scale stochastic optimisation problem. Rather than first generating labelled data and then training separately, the method directly optimises the NN parameters together with

the per-scenario state and control trajectories to minimise the closed-loop cost over a fixed set of initial state scenarios. This directly targets closed-loop performance rather than open-loop imitation accuracy, and requires significantly fewer training scenarios while achieving lower closed-loop cost and fewer constraint violations than two-stage BC.

Ghezzi et al. [5] introduce the Q-loss, a training objective that embeds the optimal control value function into the IL loss. Unlike standard MSE which penalises deviation from the expert action, the Q-loss assigns a cost to each NN output by fixing it as the first control of the optimal control problem and re-solving for the optimal subsequent trajectory. This connects IL directly to the underlying optimal control problem and has been shown empirically to produce significantly fewer constraint violations than plain regression, with comparable closed-loop cost on a cart-pole benchmark.

Dataset Aggregation (Dagger) addresses distribution shift by iteratively querying the expert on states visited by the learned policy, progressively expanding the training set. Generative Adversarial IL and Inverse RL offer further alternatives but introduce significant additional complexity in training.

D. Justification for Behaviour Cloning in This Work

For the monopropellant rocket engine control problem considered here, standard BC is the appropriate choice for several reasons. First, as demonstrated by Kim et al. [2] and Park et al. [3], BC is sufficient to achieve near-expert performance in closed-loop deployment when the training distribution covers the operating envelope. Second, the NN policy in this work takes the current chamber pressure as an input, providing implicit closed-loop feedback that partially mitigates the distribution shift problem inherent to BC. Third, the system is a single-input single-output pressure regulation task with a well-defined and bounded operating range, making the expert demonstrations representative and the learned mapping relatively smooth.

The more advanced methods of Cao and Gopaluni [4] and Ghezzi et al. [5] offer improved closed-loop performance but require access to the closed-loop cost function and system dynamics during training, adding significant implementation complexity. Given the empirical evidence from the BC literature and the characteristics of this specific control problem, standard BC with a weighted MSE loss is a well-motivated and practically viable approach.

III. SYSTEM MODELLING AND CONTROL

This chapter describes the physical monopropellant rocket engine system, the two mathematical models used for simulation and control, the operating constraints, and the NMPC formulation that serves as the expert policy for BC.

A. Physical System Overview

The system under consideration is a monopropellant thruster in which a single liquid propellant (ethanol, C_2H_6O) is fed from a pressurised tank through a pipe and a proportional

valve into a combustion chamber, where it decomposes and exhausts through a converging nozzle. The flow path is:

$$\text{Tank} \rightarrow \text{Pipe} \rightarrow \text{Valve } (k_v) \rightarrow \text{Chamber} \rightarrow \text{Nozzle} \quad (1)$$

The tank is treated as an ideal pressure source at $p_{bc} = 30$ bar. The pipe introduces friction losses that depend on the Reynolds number of the flow and the pipe geometry. The proportional valve is the sole actuator; its opening is characterised by the flow coefficient $k_v \in [0.1, 1.0]$, where higher values correspond to greater flow. The combustion chamber converts propellant mass flow into heat and pressure. The nozzle operates under choked conditions, so the throat mass outflow is determined entirely by the chamber pressure and the combustion characteristic velocity c^* .

The control objective is to regulate the chamber pressure p_c to a reference $p_{c,\text{ref}}$ by commanding k_v . Chamber pressure directly determines thrust, making its precise regulation essential for consistent engine performance.

B. Full-Order Simulation Model

The full-order model captures the coupled dynamics of pipe flow, pipe pressure, valve position, and chamber pressure. The state vector is $\mathbf{x} = [\dot{m}_p, p_p, k_v, p_c]^\top$ with control input $u = k_{v,\text{set}}$, governed by the following system of four ordinary differential equations:

$$\dot{m}_p = \frac{A_p}{l_p} \left(p_{bc} - p_p - \frac{\zeta_p}{\rho_p} \dot{m}_p |\dot{m}_p| \right) \quad (2)$$

$$\dot{p}_p = \frac{a_p^2}{V_p} (\dot{m}_p - m_v) \quad (3)$$

$$\dot{k}_v = \frac{k_{v,\text{set}} - k_v}{T_v} \quad (4)$$

$$\dot{p}_c = \frac{R_{cc} T_{cc}}{V_c} (m_v - m_c) \quad (5)$$

where $\dot{m}_p$ is the pipe mass flow rate (the time derivative of pipe propellant mass), p_p is the pipe pressure, k_v is the actual valve position, and p_c is the chamber pressure. The algebraic relations completing the model are:

- **Valve mass flow:** $m_v = \sqrt{(p_p - p_c)/R_v}$, where $R_v = 3600^2 \times 10^5 / (\rho_p^2 k_v^2)$ is the valve resistance
- **Nozzle outflow:** $m_c = p_c A_{th} / c^*$ under choked flow assumption
- **Combustion velocity:** $c^* = 1537 + 0.572 p_c^{0.315}$, an empirical power law fit
- **Pipe friction:** $\zeta_p = \lambda l_p / (d_p \cdot 2A_p^2)$, using the Darcy-Weisbach relation with a Reynolds-dependent Blasius friction factor λ
- **Valve dynamics:** first-order lag with time constant $T_v = 0.1$ s

Equation (2) is the pipe momentum equation: the tank pressure drives flow against pipe pressure and quadratic friction losses. Equation (3) is a compressibility equation: pipe pressure rises when inflow exceeds valve outflow. Equation (4)

models the valve actuator as a first-order lag — the valve does not jump instantaneously to the commanded position. Equation (5) is the chamber mass balance: pressure rises when valve inflow exceeds nozzle outflow. The model is integrated numerically using Runge-Kutta 4th order (RK4) at a timestep of $h_{\text{sim}} = 0.1$ ms.

C. Reduced-Order Model

Running NMPC on the full 4-state model is computationally prohibitive. A reduced-order 1-state model is derived by applying a quasi-steady assumption to the pipe dynamics: since pipe pressure and mass flow settle on a timescale much faster than the chamber pressure, they can be treated as always at equilibrium. This eliminates m_p and p_p as dynamic states and yields an equilibrium valve value $k_{v,\text{eq}}(p_c)$ as a function of chamber pressure alone.

Using CasADi symbolic differentiation and the implicit function theorem, the reduced-order dynamics take the form:

$$\dot{p}_c = - \left. \frac{\partial g / \partial k_v}{\partial g / \partial p_c} \right|_{k_v = k_{v,\text{eq}}} \cdot \frac{k_{v,\text{set}} - k_{v,\text{eq}}}{T_v} \quad (6)$$

where $g(p_c, k_v) = 0$ is the quasi-steady constraint equation. The result is a single nonlinear ordinary differential equation (ODE) with state p_c and input $k_{v,\text{set}}$, parameterised by the current fluid and combustion properties. This model is sufficiently accurate for NMPC design while reducing the problem size by a factor of four and reducing the stiffness of the system by eliminating the fast fluid dynamics.

D. Operating Constraints

The physical constraints on the valve actuator are:

- **Valve bounds:** $k_v \in [0.1, 1.0]$
- **Rate limit:** $|\dot{k}_v| \leq 1.43 \text{ m}^3/\text{h/s}$, corresponding to a maximum change of 0.0715 per 50 ms control step
- **Pressure operating range:** $p_c \in [9.0, 15.5]$ bar

The rate limit prevents mechanical damage to the valve and limits the rate of pressure change in the chamber. These constraints are enforced directly in the NMPC optimisation.

E. NMPC Problem Formulation

The NMPC solves a finite-horizon constrained optimal control problem at every control step. Using the reduced-order model as the prediction model, the optimisation is:

$$\min_{\mathbf{u}, \mathbf{x}} \sum_{k=0}^{N-1} [Q(p_c^k - p_{c,\text{ref}})^2 + R u_k^2 + R_{\text{rate}} (\Delta u_k)^2] \quad (7)$$

where $\mathbf{u} = [u_0, \dots, u_{N-1}]^\top$ with $u_k = k_{v,k}$ is the sequence of valve commands and $\mathbf{x} = [p_c^0, \dots, p_c^N]^\top$ is the sequence of predicted states, both treated as decision variables. The optimisation is subject to the reduced-order dynamics, valve bounds, and rate constraints, with prediction horizon $N = 20$ steps and sampling time $h = 50$ ms, giving a 1 second lookahead. The weighting matrices are $Q = 1$, $R = 10^{-3}$, and $R_{\text{rate}} = 1$.

The NMPC operates in reactive mode: at each timestep, the current reference $p_{c,\text{ref}}$ is held constant across the entire prediction horizon. This eliminates the need for future reference knowledge and makes the controller directly replicable by a NN that observes only current measurements. The optimisation is solved using CasADi with the IPOPT interior point solver, requiring 10–50 ms per call — motivating the neural network approximation developed in subsequent chapters.

IV. BEHAVIOUR CLONING

Behaviour cloning (BC) is applied to learn a neural network (NN) policy that imitates the NMPC expert controller. The learned policy maps the current system state and reference directly to a valve command, enabling real-time deployment without the computational overhead of online optimisation. Throughout this work, the full-order 4-state model (`Monoprop_sim`, Section III) serves as the simulation environment, while the NMPC uses the reduced-order 1-state model internally for prediction.

A. Expert Policy: Reactive NMPC

The expert policy is a reactive NMPC controller that operates as a set-point tracker: at each timestep it solves a finite-horizon optimal control problem using only the current reference pressure, held constant across the prediction horizon. This single-step lookahead formulation produces the same input-output behaviour as a deployable controller that does not require future reference knowledge. The NMPC uses the reduced-order 1-state model of the engine and is solved via CasADi/IPOPT at a sampling rate of 50 ms.

B. Neural Network Architecture

The policy network is a fully-connected feedforward NN with the following structure:

$$\hat{k}_v = 0.1 + 0.9 \cdot \sigma(W_3 \tanh(W_2 \tanh(W_1 \mathbf{x} + b_1) + b_2) + b_3) \quad (8)$$

where $\mathbf{x} = [\bar{p}_c, \bar{p}_{c,\text{ref}}, k_{v,\text{prev}}]^\top$ is the input vector and $\sigma(\cdot)$ denotes the sigmoid function. The network has two hidden layers of 5 neurons each with tanh activations, and a sigmoid output layer that bounds $\hat{k}_v \in [0.1, 1.0]$, matching the physical valve constraints. The total parameter count is 56. This architecture was selected empirically: larger configurations (up to 10 neurons per layer) were evaluated on earlier dataset versions but showed no improvement on the final dataset, confirming that a small network is sufficient for this low-dimensional, near-linear mapping.

C. Input-Output Mapping

The NN learns the mapping:

$$f(p_c, p_{c,\text{ref}}, k_{v,\text{prev}}) \rightarrow k_{v,\text{set}} \quad (9)$$

The previous valve command $k_{v,\text{prev}}$ is included as an input because the NMPC is implemented as a set-point tracker: its optimal output at each timestep depends on the current valve position, since the valve dynamics are modelled as a first-order

lag. To accurately imitate this behaviour, the NN must observe the same information as the NMPC.

D. Data Normalisation

Pressure inputs p_c and $p_{c,\text{ref}}$ are normalised to zero mean and unit variance using statistics computed from the training dataset:

$$\bar{p} = \frac{p - \mu_p}{s_p} \quad (10)$$

The valve command $k_{v,\text{prev}}$ is used raw as it is already bounded in $[0.1, 1.0]$. Normalisation statistics are saved alongside the model weights to ensure consistent inference at deployment.

V. TRAINING DATA COLLECTION

The quality and coverage of the training dataset directly determines how well the BC policy generalises at deployment. This chapter describes the procedure used to generate the dataset of expert (state, action) pairs by running the reactive NMPC in closed-loop simulation. The closed-loop plant is the full-order 4-state simulation model (`Monoprop_sim`) described in Section III, while the NMPC uses the reduced-order 1-state model internally for prediction.

A. Episode Design

Each episode begins from a steady-state initial condition. A random initial valve opening $k_{v,\text{init}} \sim \mathcal{U}(0.40, 0.80)$ is selected, and the corresponding equilibrium state vector is computed by solving the steady-state equations of the 4-state simulation model. The episode then runs for $T = 5$ s at a sampling interval of $h = 50$ ms, giving 80 controlled steps per episode after discarding the final prediction-horizon tail.

The dataset consists of 125 such episodes, yielding a total of 10 000 labelled observation-action pairs $(p_c, p_{c,\text{ref}}, k_{v,\text{prev}}, k_{v,\text{set}})$. At each step k , the NMPC is queried with the current pressure, reference, and previous valve command, and the resulting optimal control input $k_{v,\text{set}}^*$ is recorded as the supervised label.

B. Zero-Order Hold Reference Generation

Reference trajectories are generated as zero-order hold (ZOH) step sequences to match the piecewise-constant command signals expected in practice. For each episode, the reference is initialised at the steady-state pressure corresponding to $k_{v,\text{init}}$. Step changes are introduced at random times:

- 1) The first step occurs at $t_1 \sim \mathcal{U}(0.3, 0.6)$ s.
- 2) Subsequent steps occur at intervals $\Delta t \sim \mathcal{U}(0.8, 1.5)$ s, until $t > T - 1.0$ s.

At each step, a new target $k_{v,\text{next}} \sim \mathcal{U}(k_{v,\text{min}}, k_{v,\text{max}})$ is sampled and the corresponding steady-state chamber pressure is computed. The reference value held between step changes is therefore always a physically realisable steady-state pressure.

The NMPC operates in reactive mode: at each timestep k , the current reference $p_{c,\text{ref}}(k)$ is held constant across the entire prediction horizon. This eliminates the need for future reference knowledge and makes the collected data representative of a deployable controller.

C. Operating Range and k_v Bounds

The operating pressure range is set to $[9.0, 15.5]$ bar, matching the safe operating envelope of the monopropellant thruster. The corresponding valve bounds $k_{v,\min}$ and $k_{v,\max}$ are computed by root-finding on the steady-state pressure function $p_c^{\text{ss}}(k_v)$ using Brent's method:

$$p_c^{\text{ss}}(k_{v,\min}) = 9.0 \text{ bar} \Rightarrow k_{v,\min} \approx 0.405, \quad (11)$$

$$p_c^{\text{ss}}(k_{v,\max}) = 15.5 \text{ bar} \Rightarrow k_{v,\max} \approx 0.841. \quad (12)$$

Reference step targets are constrained to this range to ensure that all training labels correspond to reachable steady states within the physical operating envelope. The initial valve command $k_{v,\text{init}}$ is sampled from $[0.40, 0.80]$, a range that overlaps with the training envelope and extends slightly below $k_{v,\min}$ to include transient conditions close to the lower boundary.

D. Parallel Data Collection

Running 125 NMPC episodes sequentially is computationally expensive due to the CasADi/IPOPT solver invoked at every timestep. Collection is therefore parallelised using Python's `multiprocessing.Pool` with 8 workers. Each worker independently initialises its own CasADi and simulation objects — shared state is not possible since CasADi symbolic graphs are not serialisable across process boundaries. This reduces total collection time to approximately 5–10 minutes on a standard desktop, compared to roughly an hour for sequential execution.

E. Dataset Statistics

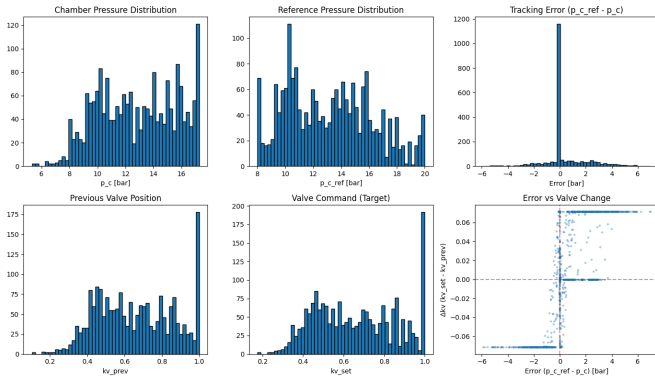


Fig. 1. Dataset distributions: chamber pressure p_c , reference pressure $p_{c,\text{ref}}$, previous valve command $k_{v,\text{prev}}$, target valve command $k_{v,\text{set}}$, tracking error, and error vs valve change scatter.

Figure 1 shows the distributions of all four variables in the collected dataset. The chamber pressure and reference pressure distributions are approximately uniform across the operating range $[9.0, 15.5]$ bar, confirming broad coverage of the operating envelope. The valve command distribution reflects the near-linear k_v - p_c relationship. The tracking error distribution is sharply peaked near zero, indicating that the NMPC successfully regulates pressure throughout data collection.

The final dataset saved to `bc_dataset_v6.npz` contains:

- 10 000 samples across 125 episodes (80 steps each)
- Chamber pressure $p_c \in [9.0, 15.5]$ bar (steady-state range)
- Reference pressure $p_{c,\text{ref}}$ drawn from the same range
- Previous valve command $k_{v,\text{prev}}$ within the collected operating range
- Target valve command $k_{v,\text{set}}$ produced by the NMPC expert

The dataset is split 80/20 into training and test sets at the *episode* level to prevent data leakage; the split strategy is described in detail in Section VI-D.

VI. NEURAL NETWORK TRAINING

This chapter describes the supervised training procedure used to fit the policy network to the collected NMPC demonstrations. The training objective, optimiser configuration, regularisation strategy, and data split procedure are each discussed in turn.

A. Loss Function: Weighted MSE

A standard MSE loss treats all timesteps equally. In the collected dataset, however, the majority of steps correspond to near-steady-state operation where $k_{v,\text{set}} \approx k_{v,\text{prev}}$ (idle steps), while a smaller fraction involve meaningful control transitions (active steps). Training with uniform MSE biases the network towards reproducing steady-state behaviour and underrepresents the transient response that is most critical for tracking.

To address this imbalance, a weighted MSE loss is defined:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N w_i \left(\hat{k}_{v,i} - k_{v,i}^* \right)^2 \quad (13)$$

where the per-sample weight is:

$$w_i = \begin{cases} 5.0 & \text{if } |k_{v,i}^* - k_{v,\text{prev},i}| \geq 0.02 \\ 1.0 & \text{otherwise} \end{cases} \quad (14)$$

Active steps — those where the NMPC commands a change of at least 0.02 in valve position — are assigned five times the weight of idle steps. This encourages the network to accurately replicate control transitions without discarding the steady-state distribution.

B. Optimiser and Learning Rate Schedule

The network is trained using the Adam optimiser with an initial learning rate of $\eta_0 = 10^{-3}$. A ReduceLROnPlateau scheduler monitors the test loss and halves the learning rate whenever no improvement is observed for 30 consecutive epochs (reduction factor = 0.5). This allows aggressive initial learning while preventing oscillation near convergence. Training runs for a maximum of 2000 epochs with a mini-batch size of 32.

C. Early Stopping

To prevent overfitting, early stopping is applied with a patience of 100 epochs. The best model state — defined as the epoch achieving the lowest test loss — is checkpointed throughout training. If no improvement is observed for 100 consecutive epochs, training terminates and the best weights are restored. The saved checkpoint includes both the model parameters and the input normalisation statistics (μ_{p_c} , s_{p_c} , $\mu_{p_{ref}}$, $s_{p_{ref}}$) required for consistent inference.

D. Train / Test Split Strategy

The dataset is divided into training (80%) and test (20%) subsets by splitting at the *episode* level rather than at the individual sample level. Splitting by sample would place consecutive timesteps from the same episode into both sets, leading to data leakage: the model could achieve low test loss simply by memorising episode-level patterns seen during training. Episode-level splitting ensures that the test set contains only trajectories never seen during training, providing a reliable estimate of generalisation performance.

With 125 episodes, the split yields 100 training episodes (8000 samples) and 25 test episodes (2000 samples). The split is performed by random shuffling of episode indices without stratification, as the episode initialisation procedure already ensures uniform coverage of the operating envelope by design — each episode samples $k_{v,init} \sim \mathcal{U}(0.40, 0.80)$ independently, so both subsets inherit approximately the same pressure and valve command distributions.

VII. RESULTS

This chapter presents the empirical evaluation of the trained BC policy. Results are reported across four dimensions: supervised training performance, open-loop prediction accuracy, closed-loop pressure tracking against the NMPC expert, and robustness to out-of-distribution conditions.

A. Training Curves

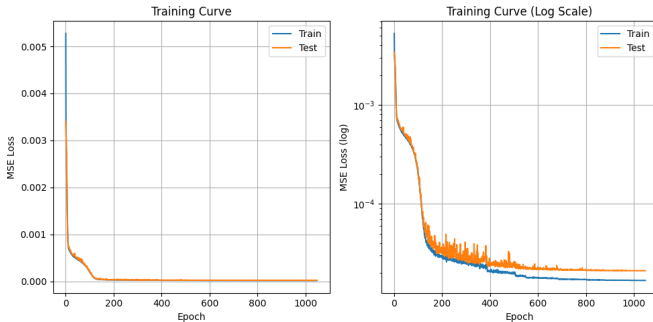


Fig. 2. Training and test loss curves (linear and log scale). Train and test losses track closely throughout, indicating no overfitting.

Training converges within approximately 300–500 epochs before early stopping triggers. Both training and test losses decrease monotonically in the early phase and track closely throughout, indicating that the 80/20 episode-level split is

sufficient to prevent overfitting for this dataset size and model capacity. The learning rate schedule reduces η one or two times during training as the test loss plateaus, each reduction producing a brief further decrease before convergence. The final weighted MSE on the test set is on the order of 10^{-4} . The unweighted MSE on the held-out test episodes is 4.0×10^{-6} (RMSE: 2.0×10^{-3} valve units).

B. Prediction Accuracy

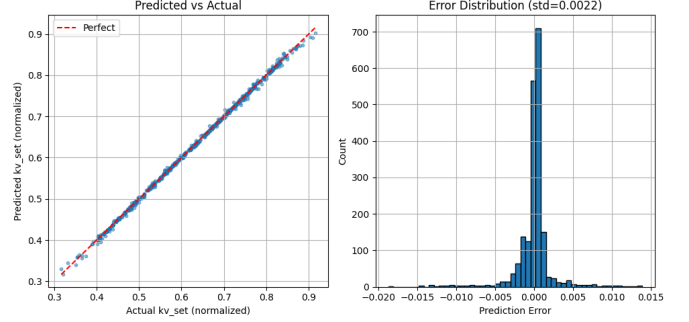


Fig. 3. Open-loop prediction accuracy on the held-out test set. Left: predicted vs actual k_v scatter plot. Right: error distribution with standard deviation of 0.0022.

Open-loop prediction accuracy is assessed on the held-out test episodes by comparing the NN output $\hat{k}_v$ to the NMPC ground truth k_v^* for each input tuple. The predicted versus actual scatter plot shows tight clustering around the identity line across the full operating range $k_v \in [0.1, 1.0]$, confirming that the network has learned a good approximation of the expert mapping.

The prediction error distribution is approximately zero-mean and symmetric, with a standard deviation of 0.0022 valve units. Larger errors are localised to the boundaries of the valve range, where the sigmoid output layer introduces slight compression, and to active-step transitions where the NMPC response is most sensitive to exact pressure conditions.

C. Closed-Loop Validation vs NMPC

Figure 4 shows the closed-loop pressure tracking results. Validation is performed by running both the NN policy and the reactive NMPC on three randomly initialised episodes with ZOH reference trajectories. In each episode, the NN receives only the current pressure measurement, the current reference step, and the previous valve command — identical information to the NMPC.

The NN tracks the reference pressure with performance comparable to the reactive NMPC across all three episodes. Step response times and steady-state errors are similar between the two controllers. Small deviations occur immediately after reference step changes, where the NN may produce a marginally different transient, but the pressure converges to the reference within 0.5–1.0 s in both cases. The NN response is smooth due to the $\tanh$ activations and the rate-limit enforcement applied post-inference.

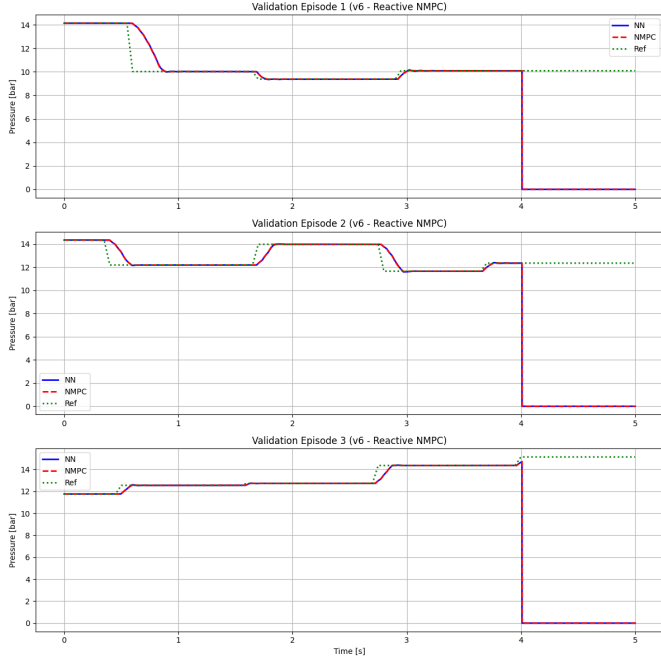


Fig. 4. Closed-loop pressure tracking: NN (blue) vs reactive NMPC (red dashed) vs reference (green dotted) across three random validation episodes.

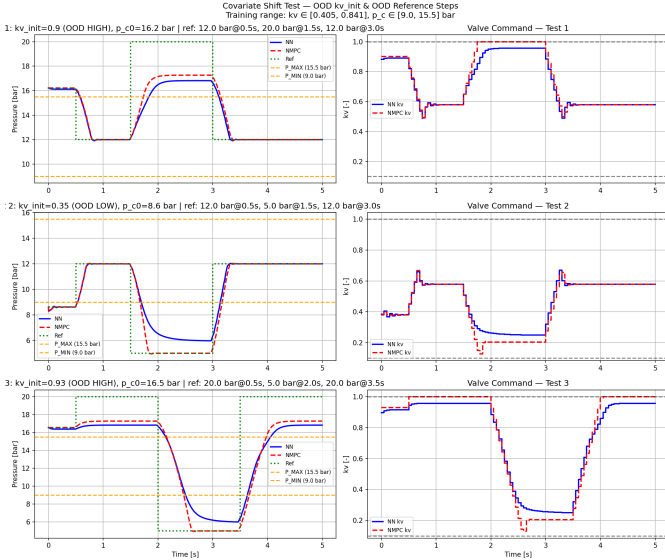


Fig. 5. OOD testing: NN vs NMPC under out-of-distribution initial valve positions and reference pressure steps outside the training range [9.0, 15.5] bar.

D. Out-of-Distribution Testing

To assess robustness, the NN is tested under conditions outside the training distribution: initial valve commands $k_{v,init} \notin [0.405, 0.841]$ and reference pressure steps outside the [9.0, 15.5] bar training range. These represent OOD scenarios that would arise if the policy were deployed beyond its intended operating envelope.

Two behaviours are observed. First, for moderately OOD initial conditions where the system transient remains within the training range, the NN self-corrects via the p_c feedback input. The network receives the measured chamber pressure as an input, which implicitly provides disturbance and initialisation error rejection without any explicit integral action. Second, for strongly OOD references (e.g. targets at 20 bar or 5 bar), the normalised pressure inputs fall far outside the training distribution, driving the $\tanh$ neurons into saturation. In this regime, the NN produces a fixed extreme valve command rather than tracking the reference. The reactive NMPC is equally unable to track these extreme references, as they lie outside its own operating constraints. Both controllers therefore degrade similarly, and the NN does not introduce additional failure modes beyond those inherent to the system design.

E. Computational Performance

The primary motivation for replacing NMPC with a NN policy is computational efficiency. At inference time, the NN requires a single forward pass through a 56-parameter network: three matrix-vector multiplications, two $\tanh$ activations, and one sigmoid. On an AMD Ryzen 5 5500U CPU, this executes in under 1 ms (mean: 0.32 ms, std: 0.07 ms, min: 0.27 ms, max: 0.98 ms). In contrast, the reactive NMPC invokes the CasADi/IPOPT solver at every 50 ms step, which requires iterative nonlinear programme convergence and typically takes 54–456 ms per call on the same hardware (mean: 60.6 ms, std: 23.8 ms). The NN therefore achieves a speedup of approximately two orders of magnitude, making real-time deployment on resource-constrained embedded hardware feasible.

VIII. DISCUSSION

This chapter interprets the results presented in Chapter VII and discusses the underlying reasons for the observed behaviour, the limitations of the approach, and directions for future improvement.

A. Covariate Shift Analysis

Covariate shift is the central theoretical concern with BC: the training distribution of states is determined by the expert trajectory, but at deployment the policy visits states determined by its own closed-loop behaviour. If the policy makes a small error that moves the system to a state not seen during training, subsequent errors may compound, leading to trajectory divergence.

In this work, covariate shift is mitigated by two structural properties of the system. First, the NN takes the current measured chamber pressure p_c as an input. This means the

policy has direct access to the consequence of any previous error — if the pressure deviates from the reference due to a suboptimal valve command, the next input to the network already reflects this deviation. The network therefore implicitly closes a feedback loop without any explicit integral action, providing self-correction for slow disturbances and moderate initialisation errors.

Second, the rate limit of $|\Delta k_v| \leq 0.0715$ per step bounds the magnitude of any single erroneous action. Even in the worst case, the policy cannot drive the system away from the training distribution faster than the system dynamics allow recovery. These two factors explain why the closed-loop validation results show near-NMPC performance without any iterative data aggregation or online expert queries, consistent with the findings of Kim et al. [2] and Park et al. [3] in similarly structured control problems.

For strongly OOD inputs — references outside the [9.0, 15.5] bar training range — the $\tanh$ activations saturate, producing a fixed extreme valve command. While this is a failure of tracking, it is not a dangerous failure: the valve is clipped to its physical bounds, and the rate limit prevents aggressive transients. The system settles at the nearest reachable steady state rather than diverging.

B. Limitations of Behaviour Cloning

Despite the strong closed-loop performance observed, BC carries inherent limitations that should be acknowledged. The trained policy is only as good as the expert: any systematic bias in the NMPC solutions — due to solver tolerances, model mismatch, or suboptimal cost weights — is directly inherited by the NN. Unlike RL, BC has no mechanism to discover better-than-expert behaviour.

Furthermore, the performance guarantee of BC is open-loop: the network is trained to minimise imitation error on the training distribution, not closed-loop tracking error. The advanced methods discussed in Chapter II directly address this gap and represent natural upgrade paths.

If distribution shift becomes a practical concern — for example when the engine is operated under unmodelled disturbances such as tank pressure drop or combustion instabilities — iterative data aggregation methods could be applied to expand the training set with expert labels at states visited by the learned policy.

C. Effect of Network Size

The policy network used in this work is intentionally small: two hidden layers of 5 neurons each, totalling 56 parameters. This choice is motivated by the near-linear relationship between k_v and p_c at steady state, which means the expert mapping is relatively smooth and low-dimensional. The small network size has practical advantages beyond computational efficiency: it is less prone to overfitting on the 10 000-sample dataset, and its inference time is deterministic and extremely short.

A larger network could in principle capture finer details of the NMPC policy near rate-saturated transitions, but the

closed-loop validation results suggest that the current architecture is already sufficient for the operating envelope. The diminishing returns of increasing network size in this application are consistent with the findings of Lee and Kang [1], who similarly found that a small feedforward network is adequate for approximating NMPC in a smooth, bounded control task.

D. k_v - p_c Mapping Linearity

The steady-state relationship between valve command k_v and chamber pressure p_c is approximately monotone and nearly linear within the operating range [9.0, 15.5] bar. This has two important implications for the BC approach. First, it means the input-output mapping the network must learn is well-conditioned: small changes in the input produce predictable changes in the output, and there are no sharp non-linearities or discontinuities to approximate. Second, the near-linearity justifies the use of a small network with $\tanh$ activations, which can represent near-linear functions efficiently without requiring deep or wide architectures.

Outside the training range, the mapping becomes more nonlinear — c^* changes more rapidly with pressure, and the pipe friction coefficient enters a different flow regime. This is consistent with the observed $\tanh$ saturation behaviour in OOD testing, where the network produces fixed extreme outputs rather than interpolating meaningfully beyond the training envelope.

IX. CONCLUSION

This work has demonstrated that BC is an effective and practical method for approximating a reactive NMPC controller for monopropellant rocket engine pressure regulation. A compact feedforward NN with 56 parameters was trained on 10 000 expert demonstrations collected from closed-loop NMPC simulations, and validated against the expert on unseen reference trajectories.

The trained policy achieves closed-loop pressure tracking performance comparable to the reactive NMPC across the operating range of [9.0, 15.5] bar, with step response times and steady-state errors that closely match the expert. The computational gain is significant: inference takes under 1 ms compared to 10–50 ms for the NMPC solver, representing a speedup of approximately two orders of magnitude. This makes real-time deployment on embedded hardware feasible without sacrificing control quality within the training envelope.

The analysis of covariate shift revealed that the inclusion of the measured chamber pressure as a network input provides implicit closed-loop feedback, enabling the policy to self-correct moderate deviations without requiring iterative expert queries or RL fine-tuning. The physical rate limit on the valve further bounds the impact of any single erroneous action, contributing to the overall robustness of the approach.

A. Future Work

Several directions remain open for extending this work. First, the training data could be enriched with process noise,

measurement uncertainty, and unmodelled disturbances such as tank pressure drop during burn, to improve robustness under realistic operating conditions. Second, the weighted MSE loss could be replaced with the Q-loss proposed by Ghezzi et al. [5], which directly optimises closed-loop performance and more closely targets deployment behaviour than open-loop imitation accuracy. Third, a cascaded IL and RL approach could be applied: the BC policy provides a strong initialisation, and RL fine-tunes it through environment interaction to recover from distribution shift and improve beyond the expert [6]. Finally, the approach could be extended to a bipropellant engine with two actuators and an oxidiser-to-fuel ratio constraint, where the increased complexity makes NMPC approximation even more valuable.

REFERENCES

- [1] T. Lee and Y. Kang, "Performance Analysis of Deep Neural Network Controller for Autonomous Driving Learning from a Nonlinear Model Predictive Control," *Electronics*, vol. 10, no. 7, p. 767, 2021.
- [2] S. Kim, K. Han, and S. B. Choi, "Imitation Learning of Nonlinear Model Predictive Control for Emergency Collision Avoidance," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 2908–2918, 2024.
- [3] S. Park, D. G. Nguyen, J. Park, D. Kim, J. S. Eo, and K. Han, "Deep Neural Network NMPC for Computationally Tractable Optimal Power Management of Hybrid Electric Vehicles." arXiv:2403.11104, 2024.
- [4] D. Cao and R. B. Gopaluni, "Deep Neural Network Approximation of Nonlinear Model Predictive Control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 4091–4096, 2020.
- [5] A. Ghezzi, J. Hoffman, J. Frey, J. Boedecker, and M. Diehl, "Imitation Learning from Nonlinear MPC via the Exact Q-Loss and its Gauss-Newton Approximation," *IEEE Control Systems Letters*, vol. 7, pp. 2689–2694, 2023.
- [6] K. Dresia, S. Gonschorek, G. Waxenegger-Wilfing, J. Deeken, and M. Oschwald, "Rocket Engine Control with Deep Reinforcement Learning," *Acta Astronautica*, 2025.

ACRONYMS

BC	behaviour cloning
IL	imitation learning
MSE	mean squared error
NMPC	nonlinear model predictive control
NN	neural network
OOD	out-of-distribution
RK4	Runge-Kutta 4th order
RL	reinforcement learning
ZOH	zero-order hold